

The 20 tables

What each table holds, what one row means, and how big it is — the fund database on Amazon RDS

The database: a fictitious fund house, **Sahyadri Mutual Fund**, with 40 schemes, over five years (1 Sep 2021 to 31 Aug 2026). **1,132,667 rows, about 210 MB**. Every number is created by code from a fixed seed, so the same data comes out every run. No model creates any data.

Two databases on one RDS instance: `mf_data` holds these 20 tables and is **read-only** for the agent. `agent_meta` holds the agent's own notes, links and embeddings.

Group A: The fund house — what the company sells (9 tables)

#	Table	One row is	Key columns	Rows	Size
1	<code>scheme_categories</code>	One SEBI category	<code>category_code</code> , <code>category_name</code> , <code>asset_class</code> , <code>risk_level</code>	36	64 kB
2	<code>benchmarks</code>	One index a scheme is compared against	<code>benchmark_name</code> , <code>benchmark_type</code> , <code>provider</code>	15	48 kB
3	<code>benchmark_values</code>	One index on one trading day	<code>benchmark_id</code> , <code>value_date</code> , <code>index_value</code>	19,560	1.4 MB
4	<code>schemes</code>	One fund	<code>scheme_code</code> , <code>scheme_name</code> , <code>category_id</code> , <code>benchmark_id</code> , <code>launch_date</code>	40	64 kB
5	<code>scheme_plans</code>	One version of a fund: Direct or Regular x Growth or IDCW	<code>scheme_id</code> , <code>plan_type</code> , <code>option_type</code> , <code>isin</code> , <code>expense_ratio_pct</code>	150	96 kB
6	<code>fund_managers</code>	One person who runs funds	<code>manager_name</code> , <code>experience_years</code> , <code>qualification</code>	25	32 kB
7	<code>scheme_manager_assignments</code>	One manager on one scheme for one period	<code>scheme_id</code> , <code>manager_id</code> , <code>manager_role</code> , <code>start_date</code> , <code>end_date</code>	74	32 kB
8	<code>nav_history</code>	The price of one plan on one day	<code>plan_id</code> , <code>nav_date</code> , <code>nav</code>	189,482	12 MB
9	<code>scheme_aum_monthly</code>	The money in one scheme at one month end	<code>scheme_id</code> , <code>month_end_date</code> , <code>aum_amount</code> , <code>folio_count</code>	2,320	224 kB

Worth knowing: an empty `end_date` in table 7 means that manager is still running the scheme. Table 9 is **calculated** from transactions, not invented, which is why its totals match the holdings exactly.

Group B: The market — what the funds invest in (6 tables)

#	Table	One row is	Key columns	Rows	Size
10	<code>sectors</code>	One industry	<code>sector_code</code> , <code>sector_name</code>	25	64 kB
11	<code>issuer_groups</code>	One business group	<code>group_name</code>	60	48 kB
12	<code>issuers</code>	One company, bank or government that issues securities	<code>issuer_name</code> , <code>group_id</code> , <code>sector_id</code> , <code>issuer_type</code>	800	192 kB
13	<code>securities</code>	One share, bond or short-term paper	<code>security_code</code> , <code>isin</code> , <code>issuer_id</code> , <code>security_type</code> , <code>maturity_date</code>	1,500	352 kB
14	<code>credit_rating_history</code>	One rating given to a bond on one date	<code>sec_cd</code> , <code>rtg_agcy</code> , <code>rtg</code> , <code>rtg_dt</code> , <code>outlk</code>	1,973	296 kB
15	<code>portfolio_holdings</code>	One security in one scheme at one month end	<code>scheme_id</code> , <code>security_id</code> , <code>as_of_date</code> , <code>market_value</code> , <code>weight_pct</code>	111,072	16 MB

Worth knowing: table 14 uses **old-style column names** on purpose: `rtg_dt` is the rating date, `outlk` the outlook, `sec_cd` the security code. Real client databases are full of these, and the system has to work them out. In table 15, `weight_pct` adds up to 100 per scheme per month, and `market_value` adds up to the scheme's AUM.

Group C: The investors — who buys the funds (4 tables)

#	Table	One row is	Key columns	Rows	Size
16	distributors	One agent or platform that sells schemes	arn_no, dist_nm, dist_typ, city_nm, actv_flg	600	128 kB
17	investors	One person who invests	pan, full_name, email, city, city_tier, kyc_status	40,000	7.4 MB
18	folios	One investor's account in one scheme	folio_number, investor_id, scheme_id, bank_account_masked	57,776	9.9 MB
19	sip_registrations	One standing instruction to invest monthly or quarterly	folio_id, plan_id, sip_amount, frequency, sip_status, distributor_arn	24,000	3.4 MB

Personal data is masked as it is created, so no realistic PAN, email, mobile or bank number exists anywhere: PAN looks like ABCPX****F. Those columns are listed in `config/pii.yaml`, are never sent to the model, and are never embedded. Table 16 also uses old-style names: `arn_no`, `dist_nm`, `actv_flg`.

Group D: The transactions — what actually happened (1 table)

#	Table	One row is	Key columns	Rows	Size
20	transactions	One purchase, SIP payment, redemption or switch	transaction_id, txn_date, folio_id, plan_id, txn_type, amount, units, nav, txn_status, arn_code	683,159	142 MB

Two-thirds of the whole database. `txn_type` is one of **Purchase, SIP, Switch In** (money coming in) or **Redemption, Switch Out** (money going out); `amount` is always positive, and the type carries the direction. `txn_status` is Completed or Rejected.

Split into 60 monthly pieces, one per month of the five years, so a query filtering on a date reads only the months it needs. On RDS you pay for the input and output a query causes, which is why the agent refuses any query on this table without a real date range.

The three hidden links

These connections exist in the data but are **never declared** as foreign keys. The metadata pipeline had to find them by matching column names and checking whether the values really overlap. All three were found, with 100% of values matching.

Link	Why it is hard
<code>scheme_aum_monthly.scheme_id</code> → <code>schemes.scheme_id</code>	Same column name, but no foreign key declares it
<code>credit_rating_history.sec_cd</code> → <code>securities.security_code</code>	Completely different names. 780 distinct values, all matching
<code>transactions.arn_code</code> → <code>distributors.arn_no</code>	Different names again. 574 distinct values, all matching

The other 19 links are declared foreign keys, so the database itself points them out.

The six stories planted in the data

Built in on purpose, so demo questions have real answers to find.

#	What is in the data	The question it answers
1	A Vardhaman Housing Finance bond is cut AA → AA- → BBB- on 12 Mar 2024; redemptions spike in the two schemes holding it	Why did net flows in the Credit Risk Fund drop in March 2024?
2	Mid Cap Fund holds about 24% across five Trident Group companies	Which scheme has the most exposure to one business group?
3	Focused Fund changes manager on 3 Oct 2023, then starts beating its benchmark	Did performance change after the manager changed?
4	SIP cancellations in tier-3 cities rise 3.5x over the last 12 months	Where are SIP cancellations rising?
5	Konkan Wealth Partners (ARN-104417) rejects about 17% of transactions, against about 2% elsewhere	Which distributor has the highest rejection rate?
6	B&FS Fund holds 12.4% in Malabar Commercial Bank from 31 Mar 2026	Is any scheme above the 10% single-issuer limit?

The answers are recorded in `evals/answer_key/planted_patterns.yaml`, which the pipeline never reads. Only the tests do.

Two rules the data follows, and the tests that prove them

Money adds up. Units × NAV equals the amount on every transaction, and no folio ever holds fewer than zero units, because the generator checks each folio's holdings before writing a redemption.

There are two routes to AUM, and they agree exactly. `scheme_aum_monthly.aum_amount` and the sum of `portfolio_holdings.market_value` give the same figure for every scheme and month. That is deliberate: a real database usually offers more than one way to calculate the same business figure, so `config/metrics.yaml` names which one is approved and two answers can never disagree.

Building it: `apply_schema.py` creates the 20 empty tables, `dataplane.generator.run` fills them, and `tests/test_stage2_data.py` (Gate 1) checks all of the above plus every planted story. Both can be re-run at any time: the same seed reproduces identical data.